

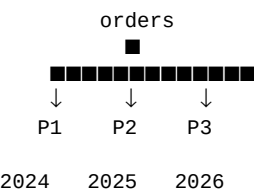
Database Partitioning — Practical System Design Notes

Hinglish • Production thinking • SQL concepts • Interview-focused

1. Partitioning Kya Hai?

Partitioning ka matlab ek large logical table ke data ko smaller partitions mein divide karna hai. Application ke perspective se ye aksar ek hi logical table hota hai, while database internally data ko separate partitions mein organize karta hai.

orders = 10 crore rows



2. Partitioning Kyun?

Very large tables mein partitioning management, maintenance, archival, and certain query patterns ko easier bana sakti hai. Agar query partition key ko restrict karti hai, database irrelevant partitions ko skip kar sakta hai.

Query:
WHERE created_at >= '2026-01-01'

Database can identify:
→ relevant 2026 partition(s)
→ skip older partitions

Concept = Partition Pruning

Partitioning har query ko automatically faster nahi banati. Query ko partitioning scheme ke saath align hona chahiye.

3. Real-Life Analogy

Socho ek warehouse mein 10 crore orders ek hi room mein rakhe hain. Instead, 2024, 2025 aur 2026 ke orders separate rooms mein rakho. Agar tumhe sirf 2026 orders chahiye, toh unnecessary rooms search karne ki zarurat nahi.

4. Partitioning vs Normal Indexing

Concept	Main idea
Index	Rows locate/filter karne ke liye extra access structure
Partitioning	Large table data ko separate partitions mein divide karna
Both together	Partition pruning + indexes within partitions can complement each other

5. Range Partitioning

Range partitioning mein values ke ranges ke basis par data divide hota hai. Datas iske common example hain.

orders

■■■ 2024-01 to 2024-12 → P1

■■■ 2025-01 to 2025-12 → P2

■■■ 2026-01 to 2026-12 → P3

Time-series, logs, transactions, events jaise workloads mein range partitioning useful ho sakti hai.

6. List Partitioning

List partitioning mein predefined values/categories ke basis par data divide kiya jata hai.

Region

■■■ Mumbai / Pune → P1

■■■ Delhi / Noida → P2

■■■ Bangalore / Chennai → P3

Exact partitioning rules database engine par depend karte hain. List partitioning tab useful ho sakti hai jab category values meaningful and reasonably stable hon.

7. Hash Partitioning

Hash partitioning mein partition key par hash function apply karke rows ko partitions mein distribute kiya jata hai.

customer_id

↓

HASH

↓

■■■■■■■■■■

P1 P2 P3 P4

Goal:

more even distribution

Hash partitioning distribution ko balance karne mein useful ho sakti hai, but range-based queries ke liye date/range partitioning jaisa natural pruning benefit nahi mil sakta.

8. Partition Pruning

Partition pruning ka matlab query optimizer ko pata chal jata hai ki kaunse partitions relevant hain, isliye unnecessary partitions ko scan nahi karna padta.

Partitions:

P1 $\rightarrow$ 2024

P2 → 2025

P3 → 2026

Query:

```
WHERE year = 2026
```

Scan:

■ P1

■ P2

■ P3

9. Partition Pruning Kab Fail/Limit Ho Sakti Hai?

Agar query partition key ko effectively constrain nahi karti, database ko multiple/all partitions inspect karne pad sakte hain. Functions, casts, expressions, implicit conversions, or complex predicates bhi pruning effectiveness ko affect kar sakte hain depending on database.

Partition key:

created at

Good:

```
WHERE created_at >= '2026-01-01'
```



relevant partitions identifiable

Less useful:

```
WHERE some_expression(created_at) = ...
```



pruning may be harder

15. Partitioning + Indexing

Partitioning aur indexing ek dusre ke alternatives nahi hain. Large partition ke andar suitable index query ko further optimize kar sakta hai.

```
orders
■■■ 2025 partition
■    ■■■ index(customer_id)
■
■■■ 2026 partition
    ■■■ index(customer_id)
```

Query:
2026 + customer_id
→ prune old partitions
→ use index in relevant partition

16. Partitioning + Archival

Time-based partitions ka operational advantage ye hai ki old data ko archive/delete/manage karna easier ho sakta hai. Some database systems support partition-level maintenance operations that can be much more efficient than row-by-row deletes.

2026 → active
2025 → warm
2024 → archive
older → retention/delete

Partition-level lifecycle
can simplify large-table maintenance.

17. Partitioning + Transactions

Transactions multiple partitions ko touch kar sakti hain. Partitioning automatically transactions ko simpler nahi banati. Cross-partition operations ka cost and behavior database engine par depend karta hai.

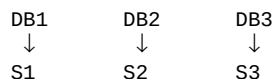
Interview mein bolo: 'Partitioning performance/maintenance technique hai; transaction semantics database engine provide karta hai and cross-partition operations need careful design.'

18. Partitioning vs Sharding

Partitioning:



Sharding:



Key difference:

Partitioning → data split within a logical DB system

Sharding → data distributed across DB nodes

Terminology different databases mein slightly different ho sakti hai, so interview mein product/context clarify karna useful hai.

19. Partitioning vs Read Replicas

Technique	Main purpose
Partitioning	Large table ko smaller logical pieces mein divide karna
Read replicas	Read workload ko multiple DB instances par distribute karna
Both	Large read-heavy systems mein together use ho sakte hain

20. Partitioning vs Sharding vs Replication

Partitioning:
ONE logical DB
→ data split into partitions

Sharding:
MULTIPLE DB nodes
→ data split across nodes

Replication:
COPY data
→ primary + replicas

They solve different problems and can be combined.

21. Real Production Example — Banking Transactions

transactions = billions of rows

Partition by transaction_date:

2024 → P1
2025 → P2
2026 → P3

Typical query:
WHERE transaction_date BETWEEN
 '2026-01-01' AND '2026-01-31'

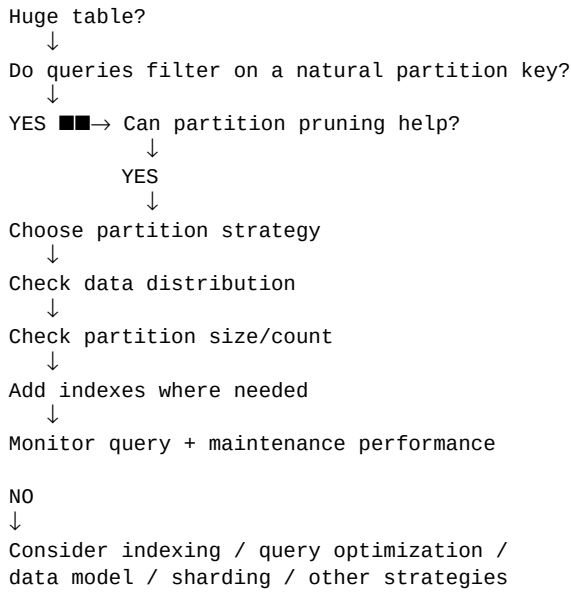
→ partition pruning
→ scan relevant partition(s)
→ index can further optimize query

Agar application ko customer_id ke basis par frequent queries bhi hain, partition strategy ko those access patterns ke saath evaluate karna hoga. One partition key rarely optimizes every query.

22. Partitioning Limitations

1. Har query faster nahi hoti.
2. Wrong partition key se poor pruning ho sakti hai.
3. Data skew / hot partitions possible hain.
4. Too many partitions management overhead create kar sakte hain.
5. Cross-partition queries expensive ho sakti hain.
6. Database-specific partitioning features/limits understand karne padte hain.
7. Partitioning alone write scaling ka universal solution nahi hai.

23. Production Decision Flow



24. Interview Q&A;

Question	Short answer
Partitioning kya hai?	Large logical table ko smaller partitions mein divide karna.
Why partition?	Query pruning, maintenance, archival and large-table management benefits.
Partition key?	Column/expression used to assign rows to partitions.
Partition pruning?	Irrelevant partitions ko query ke time skip karna.
Range partitioning?	Value ranges, commonly dates.
List partitioning?	Predefined category/list values.
Hash partitioning?	Hash-based distribution, often for more even spread.
Hot partition?	One partition gets disproportionate data/traffic.
Partitioning = sharding?	No. Partitioning is usually within a logical DB system; sharding distributes data across DB nodes.
Partitioning = replication?	No. Replication copies data; partitioning splits data.
Can partitions have indexes?	Yes, depending on DB engine; indexing and partitioning can complement each other.
Does partitioning always improve performance?	No. Benefit depends on query patterns, pruning and data distribution.
Why date partitioning?	Natural for time-range queries, retention and archival.

25. Practice Scenarios

1. Transactions table has 2 billion rows. Queries mostly filter by date. Would you consider partitioning? Why?
2. You partition by date but a query filters only by customer_id. What can happen?
3. One partition contains 80% of the data. What problem do you see?
4. Explain partition pruning using a 2024/2025/2026 example.
5. Range vs hash partitioning: when would each make sense?
6. Why doesn't partitioning automatically replace indexing?
7. Why doesn't partitioning automatically replace sharding?
8. Can read replicas and partitioning be used together?
9. What would you monitor after introducing partitions?
10. How can partitioning help data retention/archival?
11. What are the risks of creating thousands of tiny partitions?
12. Design a partition strategy for a high-volume banking transaction table.

26. Final Cheat Sheet

PARTITIONING CHEAT SHEET

Partitioning:

- one large logical table
- split into smaller partitions

Types:

- Range
- List
- Hash

Most important concept:

Partition Pruning

- query identifies relevant partitions
- irrelevant partitions skipped

Good partition key:

- aligns with common queries
- good data distribution
- manageable growth/retention

Watch:

- hot partitions
- too many partitions
- cross-partition queries
- poor pruning
- data skew

Difference:

Partitioning → split data

Replication → copy data

Sharding → distribute data across DB nodes

Golden interview line:

"Main partition key ko workload ke query pattern, data distribution aur retention requirements ke basis par choose karunga, and then verify that partition pruning actually occurs."

27. Local Practice

A large orders/transactions table create karo. Date-based partitions simulate/implement karo, EXPLAIN se partition pruning verify karo, then compare a query that filters by partition key vs one that doesn't. Uske baad indexes add karke observe karo ki partitioning aur indexing together kaise work karte hain.